

COMP101

Introduction to Programming

2025-26

Lecture-17_4

Iteration Python

nested while loops

Nested loops overview

- Loops can have other loops within the loop body ('nested loops', 'chaining')
 - for loop with nested for loop
 - for loop with nested while loop
 - while loop with nested while loop
 - while loop with nested for loop
- 'Outer loop' is the loop with another loop in its body – which is then the 'inner loop'
- Outer loops can contain several inner loops

<https://pynative.com/python-nested-loops/>

- Outer loop controls the number of iterations of the inner loop
- Each iteration of outer loop makes inner loop start and complete its execution
 - Inner loop executes and completes all of its code for each iteration of the outer loop
 - total iterations = outer loop iters * inner loop iters
 - outer loop iters = 3, inner loop iters = 7
 - Total iterations = $3 * 7 = 21$ iterations

Nested while in while

Unknown number of students may or may not submit work for grading. Can submit any amount of work

```
student = input('Students y/n: ')           #i/p=y start outer loop
while student == 'y':
    work = input ('Work to grade y/n: ')    #i/p=y start inner loop
    while work == 'y':                      #inner loop
        grade = int(input('Enter grade: ')) #inner loop
        work = input ('More work y/n: ')    #inner loop
    student = input ('Another student y/n: ') #outer loop
```

How many numbers of 8 in 9rows,38cols

342 elements – no 'break' as every element must be iterated through (there may an 8 in the last element)

62524268625363363309774646435353635355
35525336550606040302001010101939496999
29926251413243546447446746353424231940
05937271625344676765343432524242141252
42425256464600974646353434242255241525
26262624131425262534232236634242526352
42363635243635526435290243049304939393
35335428202726254625242452637373735746
35262525202912252424252452162282524462

Find number 8 in 9rows, 38 cols

342 elements – can ‘break’ when 8 is found – no need to iterate through the rest

```
62524268262536336330977464643535363535
53552533655060604030200101010193949699
92992625141324354644744674635342423194
00593727162534467676534343252424214125
24242525646460097464635343424225524152
52626262413142526253423223663424252635
24236363524363552643529024304930493939
33533548202726254625242452637373735746
3526252520291225242425245216225252446
```

while break

```
nums = 10
while nums > 0:
    nums = nums-1
    if nums == 5: #if nested in while
        break    #stop the loop when number 5 is read in
    print (nums)
print ('loop ended at number 5')
```

```
9
8
7
6
Loop ended at number 5
```

break can cause problems with many nested layers. It exits the loop it's in but can corrupt the next layer up if not careful – can cause lot of programming overheads. Also has side-effects when used with while-else.

while - continue

```
nums = 10
while nums > 0:
    nums = nums-1
    if nums == 5:
        continue #don't print number 5
    print (nums)
print ('loop excludes number 5')
```

9
8
7
6
4
3
2
1
0
Loop excludes number 5

While - else

```
number = 1
while number < 6:
    print(number)
    number += 1
else:
    print("number is now 6")
```

- else executes when loop terminates successfully – graceful degradation

While-else was a good idea at the time – but no real practical application. Better not to use it but rely on indentation after loop end to inform of loop status.
i.e. remove the bottom else and align the last print statement with the while

'level-2 nesting'

e.g. while-else and if-else(with break)

```
attempt = 3
while attempt > 0:
    password = input("Enter your password: ")
    if password == 'COMP101':
        print("Password Accepted.")
        break      #exit the loop when test =T
    else:
        attempt = attempt-1      #count down
        print("Incorrect password. Tries remaining: ", attempt)
else:
    #executes when loop terminates successfully
    print("Maximum attempts reached - access failed")
```

While-else was a good idea at the time – but no real practical application. Better not to use it but rely on indentation after loop end to inform of loop status. i.e. remove the bottom else and align the last print statement with the while

User has 3 attempts at correct password.

When 'COMP101' is input, we successfully exit the loop – no need to test for any more entries. This is managed with the 'break' statement.

While-else executes when all 3 attempts at input have been made.

While – else (Python specific)

Info only

<https://realpython.com/python-while-loop/>

```
while <expr>:  
    <statement(s)>
```

If there is a break statement in this loop, the else statement will never execute

```
else:    #else executes when the loop ends by test becoming False  
    <additional_statements>
```

- Alternative

```
while <expr>:  
    <statement(s)>
```

If there is a break statement in this loop, the unindented line will still always execute

```
<additional_statement(s)> #this code always executes after loop ends
```

- Don't use (while – else)
 - Good idea at the time - not really any practical use

While – pass

- ‘pass’ does not execute any code
- It is a ‘null operation’
 - Nothing happens when it executes
 - Useful to show where code will be written later

```
nums = 10
```

```
while nums > 0:
```

```
    nums = nums-1
```

```
    if nums == 5:
```

```
        pass
```

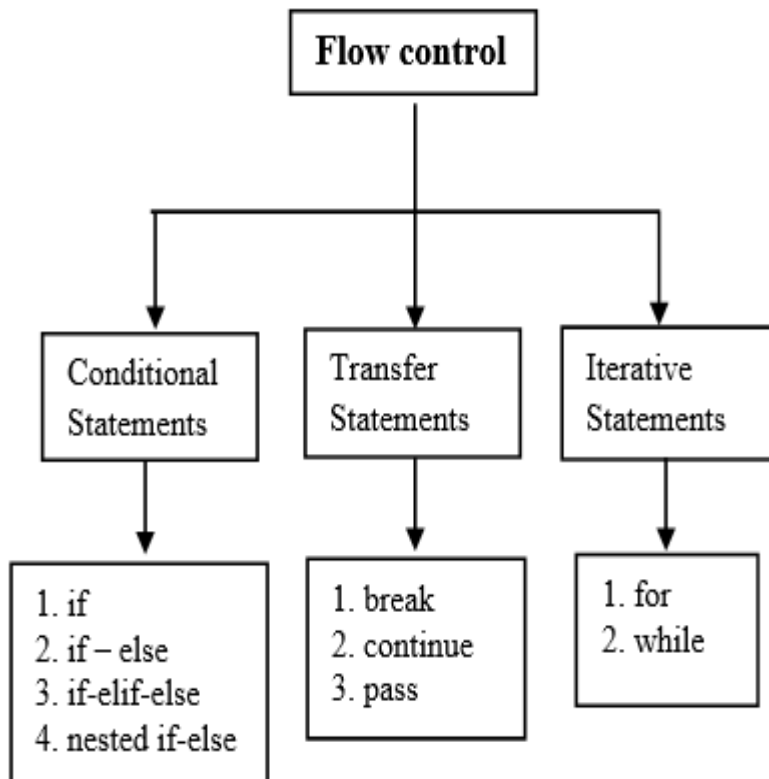
```
        print ('some code here to explain what is happening')
```

```
    print ('loop will exclude number 5 when it is finished')
```

must put some code in here
loop will exclude number 5 when it is finished

Flow control statements are divided into **three categories**

1. Conditional statements
2. Transfer statements
3. Iterative statements



Conditional statements
always evaluate T or F

Transfer statements
alter flow of execution
out of a conditional or
an iterative

Iterative statements
execute blocks of code
repeatedly as long as
condition=T

Max num levels of nesting

- Python recognises 20 levels of nesting
- Check design if using lot of nesting